

Security Audit

Noon Capital 2nd (Governance Token)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	13
Centralisation	27
Conclusion	28
Our Methodology	29
Disclaimers	31
About Hashlock	32

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Noon Capital team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Noon Capital is a DeFi governance protocol built on the Ethereum Network that allows users to stake NOON tokens and participate in decentralized governance through a veToken system. The protocol features time-locked staking with voting power multipliers, VIP staking tiers with enhanced rewards, and a comprehensive vesting mechanism for long-term token distribution.

In simple terms, users can stake NOON tokens to gain voting power in governance decisions while earning additional rewards through the vesting system, creating aligned incentives for long-term participation in the protocol.

Project Name: Noon Capital

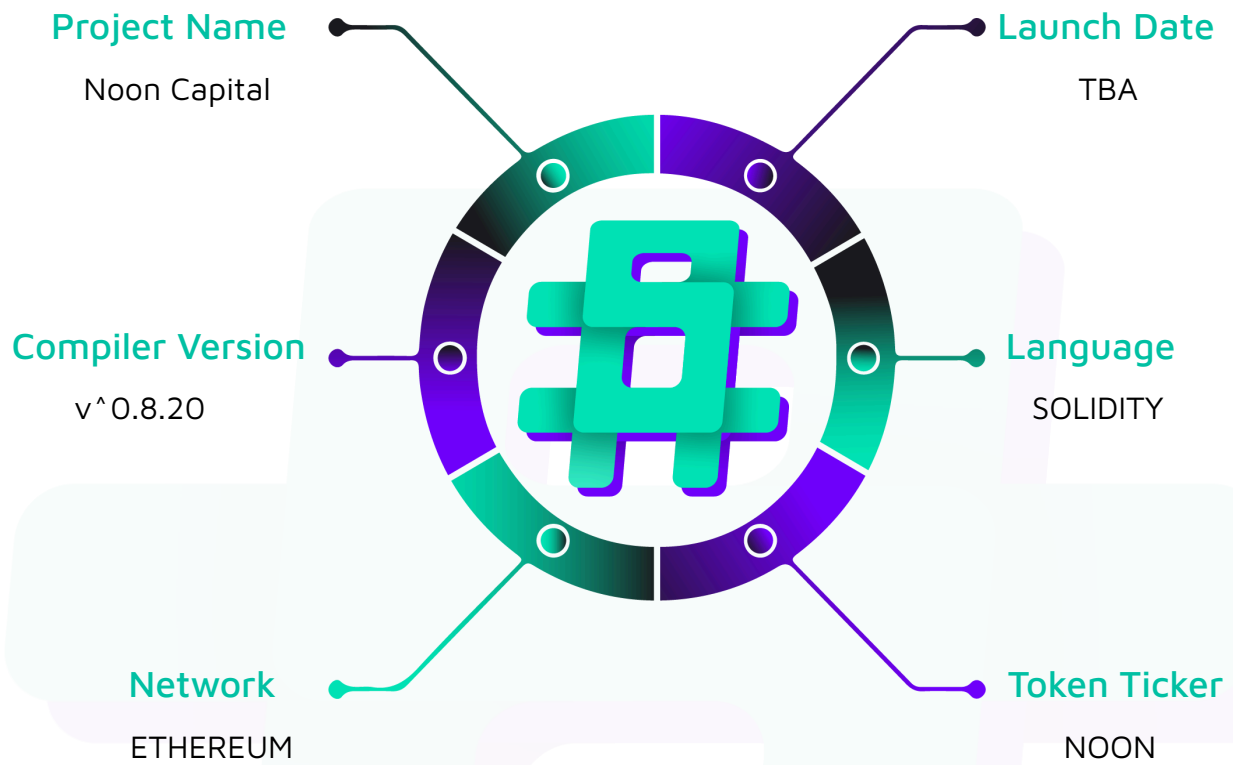
Project Type: Governance Token

Compiler Version: ^0.8.20

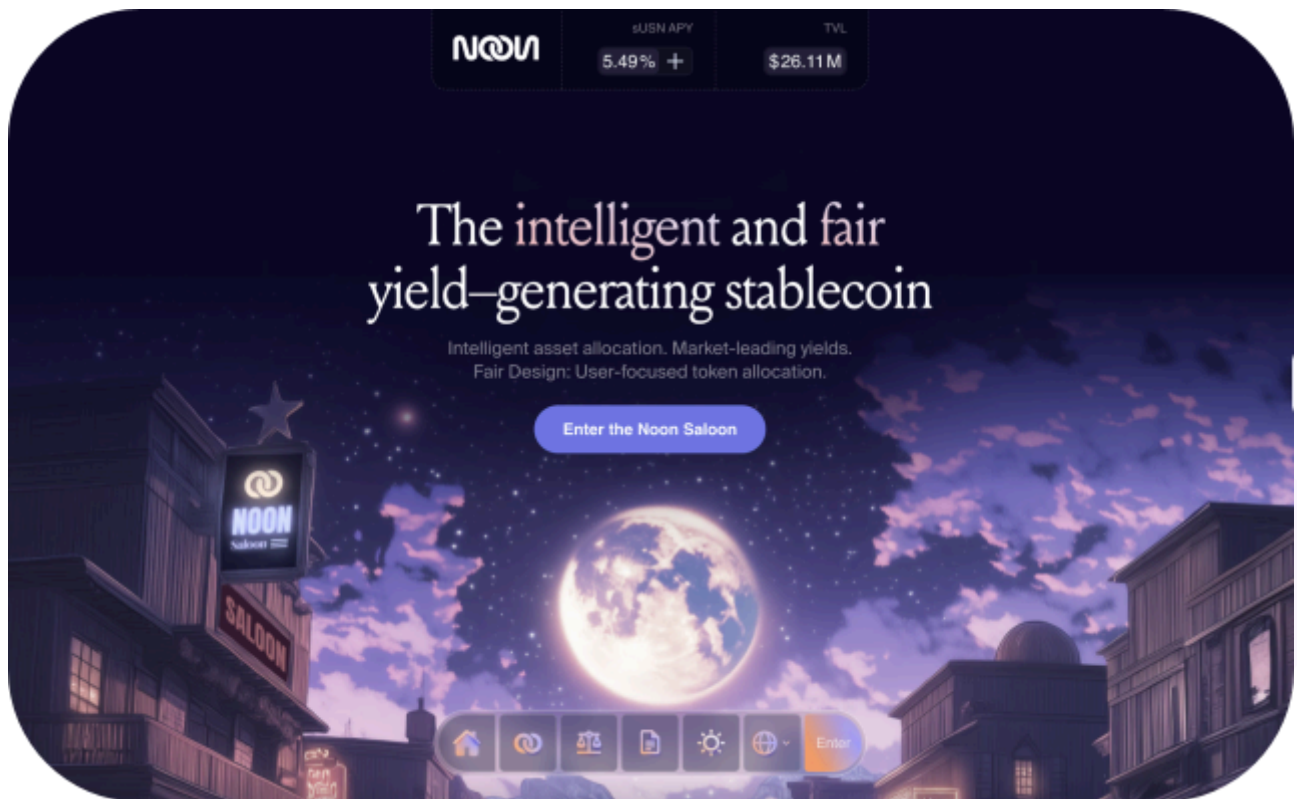
Website: <https://noon.capital/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Noon Capital project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Noon Capital Smart Contracts
Platform	Ethereum / Solidity
Audit Date	November, 2025
Contract 1	stakeNOONVesting.sol .
Contract 2	stakeNOON.sol
Contract 3	NOON.sol
Audited GitHub Commit Hash	e6ecc1858cc3c2ef8c88e29b3aa2fcc0edc969ce
Fix Review GitHub Commit Hash	8273ca28dc01ddc262c30931033d7999a581d82f

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerabilities

4 Medium severity vulnerabilities

2 Low severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
stakeNOON.sol - Allows users to lock NOON tokens to create a non-transferable NFT representing their stake, which grants them voting power in governance. - Calculates a user's voting power based on both the amount of tokens staked and the duration of the lock-up, rewarding longer commitments with a higher multiplier. - Manages the entire lifecycle of a stake, including creation, withdrawal, and extending the lock-up period to maintain or increase voting power. - Differentiates between regular stakes and special "VIP" stakes, interfacing with stakeNOONVesting.sol to create corresponding vesting schedules for VIPs.	Contract achieves this functionality.
stakeNOONVesting.sol - Acts as the dedicated vesting treasury, creating and managing reward schedules for users who make "VIP" stakes in the stakeNOON contract. - Calculates vested rewards over a 365-day period using a non-linear formula (73% cubic, 27% linear) to determine the claimable amount at any given time. - Restricts key functions to be callable only by the stakeNOON contract, ensuring that vesting schedules are only created or claimed as part of the core staking workflow.	Contract achieves this functionality.

- Requires its own pool of NOON tokens, funded by the owner, from which all vesting claims are paid out.	
NOON.sol - Functions as the core ERC20 token for the ecosystem, used for staking and rewards. - Implements a dual-permission system for transfers: a global isTransferable flag can halt most token movement, while a whitelist allows specific addresses (like core contracts) to bypass this restriction. - Includes a blacklist feature, enabling the owner to completely block certain addresses from sending or receiving tokens for security purposes. - Built as an upgradeable contract, allowing the owner to modify its rules and logic in the future without requiring a token migration.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Noon Capital project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Noon Capital project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] stakeNOON#restakePayment - The function Fails to Extend Lock-up Period, Allowing Unearned Voting Power

Description

The `restakePermanent` function in `stakeNOON.sol` grants a stake the maximum voting power multiplier (`MAX_MULTIPLIER`) by setting `isPermanent = true` but fails to update the stake's `end` timestamp in storage.

This allows an attacker to gain the voting power of a 4-year staker after locking their tokens for only the minimum duration (1 week). The attacker can then immediately withdraw their stake after influencing a governance vote, breaking the core economic principle that voting power must be proportional to the length of capital commitment.

Vulnerability Details

The `restakePermanent` function calculates a `newEnd` timestamp in memory but never assigns this value to the `stake.end` storage variable.

```
function restakePermanent(uint256 tokenId) external nonReentrant onlyTokenOwner(tokenId)
{
    Stake storage stake = stakes[tokenId];

    if (stake.amount == 0) revert StakeNotFound();

    uint256 newEnd = block.timestamp + MAX_STAKE_TIME; // 1. newEnd is calculated in
memory

    if (newEnd <= stake.end) revert NewStakeMustBeLonger();

    uint256 newMultiplier = calculateMultiplier(MAX_STAKE_TIME);

    // Update stake

    stake.multiplier = newMultiplier;
```

```
    stake.isPermanent = true;  
  
    // 2. stake.end is never updated in storage.  
  
    emit PermanentStakeRestaked(tokenId, newEnd);  
  
}
```

An attacker can first create a stake with the minimum duration of one week. After waiting for the stake to expire, they call `restakePermanent(tokenId)`. This sets the stake's multiplier to the maximum and marks it as permanent, but crucially, the `end` time remains in the past. With maximum voting power now acquired, the attacker can influence governance proposals, as the `getVotingPower` function skips the expiry check for permanent stakes. Immediately after voting, they can successfully call `withdrawWithReward(tokenId)` because the stake's end time has already passed, allowing them to exit their position.

Impact

This vulnerability allows for significant governance manipulation, as an attacker can "rent" maximum voting power for minimal cost to illegitimately influence or control critical proposals.

Recommendation

The `stake.end` storage variable must be updated to the newly calculated `newEnd` value within the `restakePermanent` function.

Status

Resolved

Medium

[M-01] stakeNOON#unstakePermanent - The function Unintentionally Re-locks User Funds

Description

The `unstakePermanent` function in `stakeNOON.sol` behaves unintentionally and acts as a trap for user funds. The function's name implies it will remove the permanent status from a stake, allowing it to be withdrawn after its original lock-up period. Instead, the function `re-locks` the user's funds for a new, full `MAX_STAKE_TIME` (4 years), starting from the moment the function is called. This behavior is directly contrary to reasonable user expectations and results in a denial of service on their own capital.

Vulnerability Details

The flaw lies in the unconditional reset of the stake's end time. Although, the comment on the code specifically states, "Set end time to current time" but the actual logic sets the end time to the `MAX_STAKE_TIME`

```
function unstakePermanent(uint256 tokenId) external nonReentrant onlyTokenOwner(tokenId)
{
    // ... checks ...

    // Remove permanent status
    stake.isPermanent = false;

    // Set end time to current time

    stake.end = block.timestamp + MAX_STAKE_TIME; // <-- VULNERABILITY: Resets lock-up to
a new 4 years

    stake.stakeDate = block.timestamp;

    emit PermanentStakeUnlocked(tokenId);
}
```


Impact

Consider a user with a "permanent" stake created two years ago, with funds scheduled to unlock in another two years. The user decides they want to exit their position when the original lock-up ends and calls `unstakePermanent(tokenId)`, logically assuming it will simply remove the permanent status. Instead, the function executes and resets `stake.end` to `block.timestamp + 4 years`. The user's funds are now unexpectedly re-locked for a new 4-year period, extending their total commitment from four years to six and causing a direct denial of service on their own capital.

Recommendation

The function's logic should be fundamentally reconsidered. A safe and logical implementation would be to simply remove the `isPermanent` status and its associated multiplier benefit, without altering the original `stake.end` time.

Status

Resolved

[M-02] stakeNOON#updateStake - The function Logically Prevents Stake Extensions

Description

The `updateStake` function contains a logical flaw that prevents users from extending their stake before it has expired. The function includes a `require(block.timestamp >= stake.end)` check, which is counter-intuitive and detrimental to the user. A user who wishes to proactively extend their lock-up to maintain their voting power is forced to wait for their stake to expire, creating a "gap" where their voting power is zero.

Vulnerability Details

The requirement that a stake be expired before extension is enforced at the beginning of the function.

```
function updateStake(...) external nonReentrant onlyTokenOwner(tokenId) {  
    Stake storage stake = stakes[tokenId];  
  
    // ...  
  
    if (block.timestamp < stake.end) revert StakeNotExpired(); // <@  
  
    // ...  
}
```

Impact

A user with an active stake that has six months remaining on its lock-up may wish to extend their commitment by another two years. When they call `updateStake`, the transaction reverts because the stake has not yet expired. This forces the user to wait six months for their stake to expire, at which point they can create a new one. However, they will have lost their previous multiplier and any associated voting power benefits in the interim. This design discourages proactive, long-term alignment from community members.

Recommendation

Remove the `StakeNotExpired` check to allow users to extend their stake at any time before expiration.

Status

Resolved



[M-03] stakeNOON#updateStake - The function Uses Incorrect Duration for Multiplier Calculation

Description

When a user extends their stake using the `updateStake` function, the new multiplier is calculated based only on the `newStakeDuration` (the extension period) rather than the total remaining lock-up time. This breaks the protocol's core economic model, as the multiplier no longer accurately reflects the user's full capital commitment.

Vulnerability Details

The multiplier calculation inside `updateStake` only considers the new duration being added.

```
// Extend duration if provided
if (newStakeDuration > 0) {
    if (newStakeDuration > MAX_STAKE_TIME) revert StakeDurationTooLong();

    uint256 newEnd = block.timestamp + newStakeDuration;

    if (newEnd <= stake.end) revert NewStakeMustBeLonger();

    // Calculate new multiplier based on total stake duration

    uint256 newMultiplier = calculateMultiplier(newStakeDuration); // <@ old Multiplier
    is lost

    // Update stake

    stake.end = newEnd;

    stake.multiplier = newMultiplier;

    emit StakeExtended(tokenId, newEnd, newMultiplier);
}
```

Impact

This flaw creates an unfair comparison between users. For example, User A, who stakes for two years in a single transaction, receives a 2x multiplier.

Now, User B stakes for one year, receiving a 1x multiplier, and then six months later extends their stake by another year. The function incorrectly calculates their new multiplier based only on the one-year extension, giving them a 1x multiplier again. Although their total lock-up is now 1.5 years from the extension point, their multiplier does not reflect this, and they are rewarded less than User A despite a similar commitment.

Recommendation

The multiplier should be calculated based on the total duration of the new lock-up period.

Status

Resolved

[M-04] stakeNOON#withdrawVip - The function Creates a Single Point of Failure, Leading to Locked Principal

Description

The `withdrawVip` function is tightly coupled with the `stakeNOONVesting.sol` contract. It attempts to claim all of a user's pending vesting schedules in a loop before returning their principal stake. If even one of these `claimVesting` calls fails for any reason (e.g., the vesting contract lacks sufficient NOON tokens to cover a claim), the entire `withdrawVip` transaction reverts. This design flaw creates a single point of failure that can be triggered by external conditions, resulting in a denial of service that locks the user's principal.

Vulnerability Details

The vulnerability is located in the `for` loop within the `withdrawVip` function, which iterates through and calls `vestingContract.claimVesting`.

```
if (address(vestingContract) != address(0)) {
    IStakeNOONVesting.VestingSchedule[] memory schedules =
    vestingContract.getVestingSchedulesForStake(tokenId);

    uint256 length = schedules.length;

    for (uint256 i = length; i > 0; i--) {
        vestingContract.claimVesting(msg.sender, i - 1); // <-- Single point of failure
    }
}
```

The `claimVesting` function in `stakeNOONVesting.sol` can revert if the contract does not hold enough NOON tokens to fulfill the claim.

Impact

A VIP user with a stake and multiple vesting schedules may find their principal trapped. If the `stakeNOONVesting` contract is partially underfunded by the project owner, it may not be able to cover all outstanding claims at once. When the user's unlock period has

passed and they call `withdrawVip`, the `for` loop begins claiming vesting schedules. If it reaches a schedule where the `claimableAmount` exceeds the vesting contract's balance, the `noon.transfer` call fails. This reverts the `claimVesting` call, which in turn reverts the entire `withdrawVip` transaction. Because the failing claim is mandatory, the user's funds are trapped until the owner tops up the vesting contract. At this point, they are rewarded less than User A despite a similar commitment.

Recommendation

Implement a logic to catch the revert and break the loop and another chance for the user to claim their remaining vested tokens, right now the logic is "claim all or none" which should be changed.

Status

Resolved

Low

[L-01] stakeNOON#getVestedAmountForStake - Unbounded Loop

Description

The `getVestedAmountForStake` function iterates over all vesting schedules for a user to calculate the total vested amount for a single `stakeId`.

Vulnerability Details

A VIP user who frequently adds to their stake via `claimAndStake` will accumulate a large number of `VestingSchedule` entries. This can cause the gas cost of calling `getVestedAmountForStake` to exceed the block gas limit, effectively rendering the view function unusable for that user.

```
// contracts/stakeNOONVesting.sol:263-275

for (uint256 i = 0; i < vestingSchedules[user].length; i++) {

    VestingSchedule storage schedule = vestingSchedules[user][i];

    if (schedule.stakeId == stakeId) {

        uint256 vestedAmount = calculateVestedAmount(

            schedule.totalAmount,

            schedule.startTime,

            schedule.endTime,

            schedule.stakeId

        );

        totalVested += vestedAmount - schedule.claimedAmount;

    }

}
```


Recommendation

The function should be modified to accept start and end index parameters. This allows external callers (both `on-chain` and `off-chain`) to iterate through the `vestingSchedules` array in discrete, gas-limited chunks, rather than all at once.

Status

Acknowledged

[L-02] stakeNOONVesting#claimVesting - The function disrupts Schedule Order

Description

The `claimVesting` function uses the "swap and pop" method to remove a vesting schedule from the `vestingSchedules` array after it has been fully claimed.

```
// contracts/stakeNOONVesting.sol:199-202  
  
if (scheduleId < vestingSchedules[user].length - 1) {  
  
    vestingSchedules[user][scheduleId] = vestingSchedules[user][vestingSchedules[user].length  
    - 1];  
  
}
```

Vulnerability Details

While this is a gas-efficient way to remove an element, it does not preserve the order of the array. When a schedule is claimed, the last schedule in the array is moved into its place. This means the `scheduleId` for other vesting schedules belonging to the same user will change unpredictably.

Recommendation

A fully claimed schedule can be left in the array, as the `claimedAmount` check already prevents any re-claiming. This preserves the stability of array indices.

Status

Acknowledged

Centralisation

The Noon Capital project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Noon Capital project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.